

SOLIDWORKS API (.NET VBA C#) — Reference (298/337): ISurface (part 1 of 2)

SOLIDWORKS API Documentation (converted from CHM)

ISurface (part 1 of 2) (Type)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ISurface Interface

[See Also](#) [Members](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) : ISurface Interface

Used as the underlying definition of a face.

NOTE: Click the **Members** link, located near the top of the topic, to see this interface's methods and properties.

Visual Basic (.NET Syntax)

Visual Basic (Declaration)

```
Public Interface ISurface
```

Visual Basic (Usage)



```
Dim instance As ISurface
```

C#

```
public interface ISurface
```

C++/CLI

```
public interface class ISurface
```

Visual Basic for Applications (VBA) Syntax

See [Surface](#).

Example

[Get B-Spline Surface Parameterization Data \(C#\)](#)
[Get B-Spline Surface Parameterization Data \(VB.NET\)](#)
[Get B-Spline Surface Parameterization Data \(VBA\)](#)
[Get Surface Type \(VBA\)](#)
[Evaluate Points on Surface \(VBA\)](#)

Remarks

This interface allows you to get the surface type and various surface definition data as well as evaluate and reverse evaluate locations on the surface.

Accessors

[IAdvancedHoleElementData::SelectionEntity](#)
[IBody2::CreateBlendSurface](#) and [IBody2::!CreateBlendSurface](#)
[IBody2::CreateBsplineSurface](#) and [IBody2::!CreateBsplineSurface](#)
[IBody2::CreateExtrusionSurface](#) and [IBody2::!CreateExtrusionSurface](#)
[IBody2::CreateNewSurface](#) and [IBody2::!CreateNewSurface](#)
[IBody2::CreateOffsetSurface](#) and [IBody2::!CreateOffsetSurface](#)
[IBody2::CreatePlanarSurface](#) and [IBody2::!CreatePlanarSurface](#)
[IBody2::CreateRevolutionSurface](#) and [IBody2::!CreateRevolutionSurface](#)
[IBody2::CreateRuledSurface](#) and [IBody2::!CreateRuledSurface](#)
[IBody2::!CreateExtrusionSurfaceDLL](#)
[IBody2::!CreatePlanarSurfaceDLL](#)
[IBody2::!CreatePsplineSurfaceDLL](#)
[IBody2::!CreateRevolutionSurfaceDLL](#)
[ICoincidentMateFeatureData::EntitiesToMate](#)
[IConcentricMateFeatureData::EntitiesToMate](#)
[IFace2::GetSurface](#) and [IFace2::!GetSurface](#)
[IHingeMateFeatureData::EntitiesToMate](#)

- [IIntersectFeatureData::GetIntersectionsTools](#)
- [ILinearPatternFeatureData::D1Axis](#)
- [ILinearPatternFeatureData::D2Axis](#)
- [ILocalLinearPatternFeatureData::D1Axis](#)
- [ILocalLinearPatternFeatureData::D2Axis](#)
- [IModeler::CreateBsplineSurface](#) and [IModeler::ICreateBsplineSurface](#)
- [IModeler::CreateConicalSurface2](#) and [IModeler::ICreateConicalSurface2](#)
- [IModeler::CreateCoonsBSurface](#)
- [IModeler::CreateCylindricalSurface2](#) and [IModeler::ICreateCylindricalSurface2](#)
- [IModeler::CreateExtrusionSurface](#) and [IModeler::ICreateExtrusionSurface](#)
- [IModeler::CreateLoftSurface](#) and [IModeler::ICreateLoftSurface](#)
- [IModeler::CreateOffsetSurface](#) and [IModeler::ICreateOffsetSurface](#)
- [IModeler::CreatePlanarSurface2](#) and [IModeler::ICreatePlanarSurface2](#)
- [IModeler::CreateRuledSurface](#) and [IModeler::ICreateRuledSurface](#)
- [IModeler::CreateSphericalSurface2](#) and [IModeler::ICreateSphericalSurface2](#)
- [IModeler::CreateSweptSurface](#)
- [IModeler::CreateToroidalSurface](#)
- [IMoveFaceFeatureData::GetEndConditionEntity](#)
- [IMoveFaceFeatureData::GetFromEntity](#)
- [IPrimaryMemberFacePlaneIntersectionFeatureData::GetIntersectingPlanesAndFaces](#)
- [IRackPinionMateFeatureData::EntitiesToMate](#)
- [IStructureSystemMemberProfile::ProfileAlignmentObject](#)
- [ISurface::Copy](#) and [ISurface::ICopy](#)
- [ISurface::GetOffsetSurfParams2](#) and [ISurface::IGetOffsetSurfParams2](#)
- [ISurfaceFlattenFeatureData::Faces](#)
- [ISweepFeatureData::EndDirectionVector](#)
- [ISweepFeatureData::StartDirectionVector](#)
- [IThreadFeatureData::GetEndConditionReference](#)
- [IThreadFeatureData::StartEntity](#)

 Access Diagram

[Surface](#)

 See Also

[ISurface Members](#)
[SolidWorks.Interop.sldworks Namespace](#)

ISurface Interface Methods

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) : ISurface Interface

For a list of all members of this type, see [ISurface members](#).

 Public Methods

Name	Description
 AddTrimmingLoop	Obsolete. Superseded by ISurface::AddTrimmingLoop2
 AddTrimmingLoop2	Creates a trimming loop out of specified surface parametric UV-curves and adds it to a list of such loops.
 Copy	Gets a copy of this surface.
 CreateNewCurve	Creates a new empty curve for the part.
 CreateTrimmedSheet	Obsolete. Superseded by ISurface::CreateTrimmedSheet4
 CreateTrimmedSheet4	Obsolete. Superseded by ISurface::CreateTrimmedSheet5
 CreateTrimmedSheet5	Creates a trimmed sheet body from this surface.

	Evaluate	Evaluates the surface, given the u and v parameters of the surface.
	EvaluateAtPoint	Evaluates a surface at the specified XYZ point.
	FindMinimumRadius	Gets the minimum radius of curvature for the selected surface.
	GetBSurfParams	Obsolete. Superseded by ISurface.GetBSurfParams2
	GetBSurfParams2	Obsolete. Superseded by ISurface.GetBSurfParams3
	GetBSurfParams3	Gets the parameterization data for a B-spline surface.
	GetClosestPointOn	Uses the X,Y,Z input point to determine the closest point on the surface.
	GetExtrusionsurfParams	Gets the parameters for the extrusion surface.
	GetIntersectCurveCount	Obsolete. Superseded by ISurface.GetIntersectCurveCount2
	GetIntersectCurveCount2	Gets the number of points for a surface-curve intersection.
	GetIntersectSurfaceCount	Gets the number of curves for a surface-surface intersection.
	GetOffsetSurfParams	Obsolete. Superseded by ISurface.GetOffsetSurfParams2
	GetOffsetSurfParams2	Gets the overall offset distance of this offset surface.
	GetProfileCurve	Gets the curve used to create the surface of revolution or extrusion surface and applies to only surface of revolution or extrusion surface.
	GetProjectedPointOn	Gets the point where the input point is projected on to this surface.
	GetRevsurfParams	Gets the parameters for the surface.
	IAddTrimmingLoop	Obsolete. Superseded by ISurface.IAddTrimmingLoop2
	IAddTrimmingLoop2	Creates a trimming loop out of specified surface parametric (UV-curves) and adds it to a list of such loops.
	ICopy	Gets a copy of this surface.
	ICreateNewCurve	Creates a new empty curve for the part.
	ICreateTrimmedSheet	Obsolete. Superseded by ISurface.ICreateTrimmedSheet4
	ICreateTrimmedSheet2	Obsolete. Superseded by ISurface.ICreateTrimmedSheet4
	ICreateTrimmedSheet4	Obsolete. Superseded by ISurface.CreateTrimmedSheet5
	Identify	Gets the type of surface.
	IEvaluate	Evaluates the surface, given the u and v parameters of the surface.
	IEvaluateAtPoint	Evaluates a surface at the specified XYZ point.
	IFindMinimumRadius	Gets the minimum radius of curvature for the selected surface.
	IGetBSurfParams	Gets the b-spline surface parameters for the surface.
	IGetBSurfParamsSize	Obsolete. Superseded by ISurface.IGetBSurfParamsSize3
	IGetBSurfParamsSize2	Obsolete. Superseded by ISurface.IGetBSurfParamsSize3
	IGetBSurfParamsSize3	Gets the allocation size necessary for Bsurface parameter data retrieval in a subsequent call to ISurface.IGetBSurfParams .
	IGetClosestPointOn	Uses the X,Y,Z input point to determine the closest point on the surface.
	IGetExtrusionsurfParams	Gets the parameters for the extrusion surface.
	IGetMakIsoCurvesCount	Gets the number of curves that represent the ISO line of a given direction.
	IGetOffsetSurfParams2	Gets the overall offset distance of this offset surface.
	IGetProfileCurve	Gets the curve used to create the surface of revolution or extrusion surface and applies to only surface of revolution or extrusion surface.
	IGetRevsurfParams	Gets the parameters for the surface.
	IIntersectCurve	Obsolete. Superseded by ISurface.IIntersectCurve2
	IIntersectCurve2	Gets a surface-curve intersection.
	IIntersectSurface	Gets a surface-surface intersection.
	IMakeIsoCurve	Creates a curve that represents the ISO line of a given surface.
	IMakeIsoCurves	Creates curves on a surface.
	IntersectCurve	Obsolete. Superseded by ISurface.IntersectCurve2
	IntersectCurve2	Gets a surface-curve intersection.
	IntersectSurface	Gets a surface-surface intersection.

	IParameterization	Gets the parameterization of the surface.
	IReverseEvaluate	Gets the UV parameters at the specified location, which must be on the surface.
	IsBlending	Gets whether the surface is a blending surface.
	IsCone	Gets whether the surface is a cone.
	IsCylinder	Gets whether the surface is a cylinder.
	IsForeign	Gets whether the surface is a foreign surface.
	IsOffset	Gets whether the surface is an offset surface.
	IsParametric	Gets whether the surface is a parametric (spline type) surface.
	IsPlane	Gets whether the surface is a planar surface.
	IsRevolved	Gets whether the surface is a revolved surface.
	IsSphere	Gets whether the surface is a sphere.
	IsSwept	Gets whether the surface is swept.
	IsTorus	Gets whether the surface is a torus.
	MakeIsoCurve	Creates an untrimmed curve using the specified surface parameter.
	MakeIsoCurve2	Creates an untrimmed curve on a surface using the specified u or v surface function parameter.
	MakeIsoCurves	Creates curves on a surface.
	Parameterization	Obsolete. Superseded by ISurface.Parameterization2 .
	Parameterization2	Gets the parameterization of the surface.
	ReverseEvaluate	Gets the UV parameters at the specified location, which must be on the surface.

[Top](#)

See Also

[ISurface Interface](#)
[SolidWorks.Interop.sldworks Namespace](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ISurface Interface Members

[See Also](#) [Properties](#) [Methods](#)

[SolidWorks.Interop.sldworks Namespace](#) : [ISurface Interface](#)

The following tables list the members exposed by [ISurface](#).

Public Properties

Name	Description
 ConeParams	Obsolete. Superseded by ISurface.ConeParams2 .
 ConeParams2	Gets the parameters of a conical surface.
 CylinderParams	Gets the parameters of a cylindrical surface.
 IConeParams	Gets the parameters of a conical surface.
 ICylinderParams	Gets the parameters of a cylindrical surface.
 IPlaneParams	Gets the parameters of a planar surface.
 ISphereParams	Gets the parameters of a spherical surface.
 ITorusParams	Gets the parameters of a toroidal surface.
 PlaneParams	Gets the parameters of a planar surface.
 SphereParams	Gets the parameters of a spherical surface.
 TorusParams	Gets the parameters of a toroidal surface.

[Top](#)

Public Methods

Name	Description
AddTrimmingLoop	Obsolete. Superseded by ISurface.AddTrimmingLoop2 .

		
	AddTrimmingLoop2	Creates a trimming loop out of specified surface parametric UV-curves and adds it to a list of such loops.
	Copy	Gets a copy of this surface.
	CreateNewCurve	Creates a new empty curve for the part.
	CreateTrimmedSheet	Obsolete. Superseded by ISurface.CreateTrimmedSheet4 .
	CreateTrimmedSheet4	Obsolete. Superseded by ISurface.CreateTrimmedSheet5 .
	CreateTrimmedSheet5	Creates a trimmed sheet body from this surface.
	Evaluate	Evaluates the surface, given the u and v parameters of the surface.
	EvaluateAtPoint	Evaluates a surface at the specified XYZ point.
	FindMinimumRadius	Gets the minimum radius of curvature for the selected surface.
	GetBSurfParams	Obsolete. Superseded by ISurface.GetBSurfParams2 .
	GetBSurfParams2	Obsolete. Superseded by ISurface.GetBSurfParams3 .
	GetBSurfParams3	Gets the parameterization data for a B-spline surface.
	GetClosestPointOn	Uses the X,Y,Z input point to determine the closest point on the surface.
	GetExtrusionsurfParams	Gets the parameters for the extrusion surface.
	GetIntersectCurveCount	Obsolete. Superseded by ISurface.GetIntersectCurveCount2 .
	GetIntersectCurveCount2	Gets the number of points for a surface-curve intersection.
	GetIntersectSurfaceCount	Gets the number of curves for a surface-surface intersection.
	GetOffsetSurfParams	Obsolete. Superseded by ISurface.GetOffsetSurfParams2 .
	GetOffsetSurfParams2	Gets the overall offset distance of this offset surface.
	GetProfileCurve	Gets the curve used to create the surface of revolution or extrusion surface and applies to only surface of revolution or extrusion surface.
	GetProjectedPointOn	Gets the point where the input point is projected on to this surface.
	GetRevsurfParams	Gets the parameters for the surface.
	IAddTrimmingLoop	Obsolete. Superseded by ISurface.IAddTrimmingLoop2 .
	IAddTrimmingLoop2	Creates a trimming loop out of specified surface parametric (UV-curves) and adds it to a list of such loops.
	ICopy	Gets a copy of this surface.
	ICreateNewCurve	Creates a new empty curve for the part.
	ICreateTrimmedSheet	Obsolete. Superseded by ISurface.ICreateTrimmedSheet4 .
	ICreateTrimmedSheet2	Obsolete. Superseded by ISurface.ICreateTrimmedSheet4 .
	ICreateTrimmedSheet4	Obsolete. Superseded by ISurface.CreateTrimmedSheet5 .
	Identify	Gets the type of surface.
	IEvaluate	Evaluates the surface, given the u and v parameters of the surface.
	IEvaluateAtPoint	Evaluates a surface at the specified XYZ point.
	IFindMinimumRadius	Gets the minimum radius of curvature for the selected surface.
	IGetBSurfParams	Gets the b-spline surface parameters for the surface.
	IGetBSurfParamsSize	Obsolete. Superseded by ISurface.IGetBSurfParamsSize3 .
	IGetBSurfParamsSize2	Obsolete. Superseded by ISurface.IGetBSurfParamsSize3 .
	IGetBSurfParamsSize3	Gets the allocation size necessary for Bsurface parameter data retrieval in a subsequent call to ISurface.IGetBSurfParams .
	IGetClosestPointOn	Uses the X,Y,Z input point to determine the closest point on the surface.
	IGetExtrusionsurfParams	Gets the parameters for the extrusion surface.
	IGetMakeIsoCurvesCount	Gets the number of curves that represent the ISO line of a given direction.
	IGetOffsetSurfParams2	Gets the overall offset distance of this offset surface.
	IGetProfileCurve	Gets the curve used to create the surface of revolution or extrusion surface and applies to only surface of revolution or extrusion surface.
	IGetRevsurfParams	Gets the parameters for the surface.
	IntersectCurve	Obsolete. Superseded by ISurface.IntersectCurve2 .

	IntersectCurve2	Gets a surface-curve intersection.
	IntersectSurface	Gets a surface-surface intersection.
	IMakeIsoCurve	Creates a curve that represents the ISO line of a given surface.
	IMakeIsoCurves	Creates curves on a surface.
	IntersectCurve	Obsolete. Superseded by ISurface.IntersectCurve2
	IntersectCurve2	Gets a surface-curve intersection.
	IntersectSurface	Gets a surface-surface intersection.
	IParameterization	Gets the parameterization of the surface.
	IReverseEvaluate	Gets the UV parameters at the specified location, which must be on the surface.
	IsBlending	Gets whether the surface is a blending surface.
	IsCone	Gets whether the surface is a cone.
	IsCylinder	Gets whether the surface is a cylinder.
	IsForeign	Gets whether the surface is a foreign surface.
	IsOffset	Gets whether the surface is an offset surface.
	IsParametric	Gets whether the surface is a parametric (spline type) surface.
	IsPlane	Gets whether the surface is a planar surface.
	IsRevolved	Gets whether the surface is a revolved surface.
	IsSphere	Gets whether the surface is a sphere.
	IsSwept	Gets whether the surface is swept.
	IsTorus	Gets whether the surface is a torus.
	MakeIsoCurve	Creates an untrimmed curve using the specified surface parameter.
	MakeIsoCurve2	Creates an untrimmed curve on a surface using the specified u or v surface function parameter.
	MakeIsoCurves	Creates curves on a surface.
	Parameterization	Obsolete. Superseded by ISurface.Parameterization2
	Parameterization2	Gets the parameterization of the surface.
	ReverseEvaluate	Gets the UV parameters at the specified location, which must be on the surface.

[Top](#)

See Also

[ISurface Interface](#)
[SolidWorks.Interop.sldworks Namespace](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ISurface Interface Properties

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) : ISurface Interface

For a list of all members of this type, see [ISurface members](#).

Public Properties

Name	Description
 ConeParams	Obsolete. Superseded by ISurface.ConeParams2
 ConeParams2	Gets the parameters of a conical surface.
 CylinderParams	Gets the parameters of a cylindrical surface.
 IConeParams	Gets the parameters of a conical surface.
 ICylinderParams	Gets the parameters of a cylindrical surface.
 IPlaneParams	Gets the parameters of a planar surface.
 ISphereParams	Gets the parameters of a spherical surface.
 ITorusParams	Gets the parameters of a toroidal surface.

 PlaneParams	Gets the parameters of a planar surface.
 SphereParams	Gets the parameters of a spherical surface.
 TorusParams	Gets the parameters of a toroidal surface.

[Top](#)

See Also

[ISurface Interface](#)
[SolidWorks.Interop.sldworks Namespace](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

AddTrimmingLoop Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : AddTrimmingLoop Method (ISurface)

Obsolete. Superseded by [ISurface::AddTrimmingLoop2](#).

NET Syntax

Visual Basic (Declaration)

```
Function AddTrimmingLoop( _
    ByVal NCrvs As System.Integer, _
    ByVal VOrder As System.Object, _
    ByVal VDim As System.Object, _
    ByVal VPeriodic As System.Object, _
    ByVal VNumKnots As System.Object, _
    ByVal VNumCtrlPoints As System.Object, _
    ByVal VKnots As System.Object, _
    ByVal VCtrlPointDbls As System.Object _
) As System.Boolean
```

Visual Basic (Usage)



```
Dim instance As ISurface
Dim NCrvs As System.Integer
Dim VOrder As System.Object
Dim VDim As System.Object
Dim VPeriodic As System.Object
Dim VNumKnots As System.Object
Dim VNumCtrlPoints As System.Object
Dim VKnots As System.Object
Dim VCtrlPointDbls As System.Object
Dim value As System.Boolean

value = instance.AddTrimmingLoop(NCrvs, VOrder, VDim, VPeriodic, VNumKnots, VNumCtrlPoints, VKnots, VCtrlPointDbls)
```

C#

```
System.bool AddTrimmingLoop(
    System.int NCrvs,
    System.object VOrder,
    System.object VDim,
    System.object VPeriodic,
    System.object VNumKnots,
    System.object VNumCtrlPoints,
    System.object VKnots,
    System.object VCtrlPointDbls
)
```

C++/CLI

```
System.bool AddTrimmingLoop(
    & System.int NCrvs,
    & System.Object^ VOrder,
    & System.Object^ VDim,
    & System.Object^ VPeriodic,
    & System.Object^ VNumKnots,
    & System.Object^ VNumCtrlPoints,
    & System.Object^ VKnots,
```

```
& System.Object^ VCtrlPointDbls  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NCrvs

VOrder

VDim

VPeriodic

VNumKnots

VNumCtrlPoints

VKnots

VCtrlPointDbls

Visual Basic for Applications (VBA) Syntax

See [Surface::AddTrimmingLoop](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

AddTrimmingLoop2 Method (ISurface)

See Also [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : AddTrimmingLoop2 Method (ISurface)

Creates a trimming loop out of specified surface parametric UV-curves and adds it to a list of such loops.

.NET Syntax

Visual Basic (Declaration)

```
Function AddTrimmingLoop2( _  
    ByVal NCrvs As System.Integer, _  
    ByVal VOrder As System.Object, _  
    ByVal VDim As System.Object, _  
    ByVal VPeriodic As System.Object, _  
    ByVal VNumKnots As System.Object, _  
    ByVal VNumCtrlPoints As System.Object, _  
    ByVal VKnots As System.Object, _  
    ByVal VCtrlPointDbls As System.Object, _  
    ByVal UvRange As System.Object _  
) As System.Boolean
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim NCrvs As System.Integer  
Dim VOrder As System.Object  
Dim VDim As System.Object  
Dim VPeriodic As System.Object  
Dim VNumKnots As System.Object  
Dim VNumCtrlPoints As System.Object  
Dim VKnots As System.Object  
Dim VCtrlPointDbls As System.Object  
Dim UvRange As System.Object  
Dim value As System.Boolean  
  
value = instance.AddTrimmingLoop2(NCrvs, VOrder, VDim, VPeriodic, VNumKnots, VNumCtrlPoints, VKnots, VCtrlPointDbls, UvRange)
```

C#

```
System.bool AddTrimmingLoop2(  
    System.int NCrvs,  
    System.object VOrder,
```

```

System.object VDim,
System.object VPeriodic,
System.object VNumKnots,
System.object VNumCtrlPoints,
System.object VKnots,
System.object VCtrlPointDbls,
System.object UvRange
)

```

C++/CLI

```

System.bool AddTrimmingLoop2(
& System.int NCrvs,
& System.Object^ VOrder,
& System.Object^ VDim,
& System.Object^ VPeriodic,
& System.Object^ VNumKnots,
& System.Object^ VNumCtrlPoints,
& System.Object^ VKnots,
& System.Object^ VCtrlPointDbls,
& System.Object^ UvRange
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NCrvs

Number of surface parametric (UV) curves constituting the loop and the size of each of the *VOrder*, *VDim*, *VPeriodic*, *VNumKnots*, *VNumControlPnts* arrays

VOrder

Array containing *NCrvs* longs (VBA), Integers (VB.NET), or ints (C#, C++) representing orders of the curves (see **Remarks**)

VDim

Array containing *NCrvs* longs (VBA), Integers (VB.NET), or ints (C#, C++) representing dimensions (2, 3, or 4) of the control points of the curves (see **Remarks**)

VPeriodic

Array containing *NCrvs* longs (VBA), Integers (VB.NET), or ints (C#, C++) representing whether the curve is periodic (1) or non-periodic (0) (see **Remarks**)

VNumKnots

Array containing *NCrvs* longs (VBA), Integers (VB.NET), or ints (C#, C++) representing number of knots on the curves (see **Remarks**)

VNumCtrlPoints

Array containing *NCrvs* longs (VBA), Integers (VB.NET), or ints (C#, C++) representing number of control points on the curves (see **Remarks**)

VKnots

Knot vector array of <TotalNumKnots> doubles, where TotalNumKnots = (TotalNumKnots + *VNumKnots*[*i*]) for *i* = 1 to *NCrvs* (see **Remarks**)

VCtrlPointDbls

Control point coordinate array of <TotalNumCPCoords> doubles, where TotalNumCPCoords = (TotalNumCPCoords + (*VDim*[*i*] * *VNumCtrlPoints*[*i*])) for *i* = 1 to *NCrvs* (see **Remarks**)

UvRange

Array of four doubles representing U Low, U High, V Low, and V High (see **Remarks**)

Return Value

True if successful in adding a trimming loop to the surface, false if not

Visual Basic for Applications (VBA) Syntax

See [Surface.AddTrimmingLoop2](#).

Example

[Create Body Using Trimmed Surfaces \(VBA\)](#)

[Create Body Using Trimmed Surfaces \(VB.NET\)](#)

[Create Body Using Trimmed Surfaces \(C#\)](#)

Remarks

The list of trimming loops is created by a previous call to one of the base surface creation functions, [IBody2::CreateRevolutionSurface](#) or [IBody2::CreatePlanarSurface](#).

The information on each UV-curve is in b-spline form (knots and control point coordinates) and is compacted into two arrays, *VKnots* and *VCtrlPointDbls*.

Order of each curve in *VOrder* is, at a minimum, 2. A second order curve is a linear curve.

For each curve in *VNumCtrlPoints*, *VOrder*, and *VNumKnots*, (NumCtrlPoints >= Order) and (NumKnots = NumCtrlPoints + Order).

Dim for each curve in *VDim* specifies the dimension of each *CtrlPointDbl* set in *VCtrlPointDbls*. This method expects 2D-polynomial (X,Y) or 2D-rational (X,Y,Weight) curves to be passed as trimming curves. However, you can set a flag so that this method accepts 3D-polynomial (X,Y,Z) or 3D-rational (X,Y,Z,Weight) trim curves. To do this, negate the absolute value of the first value in the *VDim* array. For example, if the first *VDim* value is 3, set it to -3. When you do this, 3D trim curves are expected.

If *Dim* is:

- 2, specify (X,Y) control points in each *CtrlPointDbl* set.
- -2, specify (X,Y,Z) control points in each *CtrlPointDbl* set.
- 3, specify (X,Y,Weight) control points in each *CtrlPointDbl*.
- -3, specify (X,Y,Z,Weight) control points in each *CtrlPointDbl*.

Multiplicity of a knot is the number of times it is repeated in the knot vector. There are specific rules around number of knots, knot multiplicity, and curve order. For example, if a curve's Periodic value is:

- 0 (non-periodic), there must be (NumCtrlPoints + Order) knots and the maximum multiplicity of any knot is Order.
- 1 (periodic), there must be (NumCtrlPoints + 1) knots and the maximum multiplicity of any knot is Order. If a knot has multiplicity greater than 1, repetitions must occur at the end of the knot vector.

If you do not want to specify a UV range, use null or Nothing in the UvRange array elements.

After calling this method to add a trimming loop, you can generate the trimmed surface by calling [IBody2::CreateTrimmedSurface](#). You can create a body from trimmed surfaces using [IBody2::CreateBodyFromSurfaces](#).

NOTE: It is highly recommended that you consult other resources for the mathematics behind b-splines, knots, knot multiplicity, and control points before attempting to use this method.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::AddTrimmingLoop2 Method](#)

Availability

SOLIDWORKS 2001Plus FCS, Revision Number 10.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

ConeParams Property (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ConeParams Property (ISurface)

Obsolete. Superseded by [ISurface::ConeParams2](#).

.NET Syntax

Visual Basic (Declaration)

```
ReadOnly Property ConeParams As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.ConeParams
```

C#

```
System.Object ConeParams {get;}
```

C++/CLI

```
property System.Object^ ConeParams {  
    System.Object^ get();  
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

Array of doubles describing the parameters of a conical surface

Visual Basic for Applications (VBA) Syntax

See [Surface::ConeParams](#).

Remarks

Returns an array of 8 doubles:

- *origin.x*
- *origin.y*
- *origin.z*
- *axis.x*
- *axis.y*
- *axis.z*
- *radius*
- *half angle*

The *half angle*, *origin*, and *radius* parameters are in meters.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IConeParams Property](#)
[ISurface::IsCone Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ConeParams2 Property (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ConeParams2 Property (ISurface)

Gets the parameters of a conical surface.

.NET Syntax

Visual Basic (Declaration)

```
ReadOnly Property ConeParams2 As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.ConeParams2
```

C#

```
System.Object ConeParams2 {get;}
```

C++/CLI

```
property System.Object^ ConeParams2 {  
    System.Object^ get();  
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

Array of doubles describing the parameters of a conical surface

Visual Basic for Applications (VBA) Syntax

See [Surface::ConeParams2](#).

Example

[Get Parameters of Conical Surface \(VBA\)](#)
[Get Parameters of Conical Surface \(VB.NET\)](#)
[Get Parameters of Conical Surface \(C#\)](#)

Remarks

Returns an array of 11 doubles:

- *origin.x*
- *origin.y*
- *origin.z*
- *axis.x*
- *axis.y*
- *axis.z*
- *radius*
- *half angle*
- *reference_direction.x*
- *reference_direction.y*
- *reference_direction.z*

Half angle element is in radians. Origin, radius, and reference direction elements are in meters.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IsCone Method \(\)](#)

Availability

SOLIDWORKS 2015 FCS, Revision Number 23.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

Copy Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : Copy Method (ISurface)

Gets a copy of this surface.

.NET Syntax

Visual Basic (Declaration)

```
Function Copy() As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.Copy()
```

C#

```
System.object Copy()
```

C++/CLI

```
System.Object^ Copy();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Copy of [surface](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::Copy](#).

Example

[Fill Hole in Part \(VBA\)](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::ICopy Method](#)
[ISurface::CreateNewCurve Method](#)
[ISurface::CreateNewCurve Method](#)
[ISurface::IMakeIsoCurve Method](#)
[ISurface::IMakeIsoCurves Method](#)
[ISurface::MakeIsoCurve Method](#)
[ISurface::MakeIsoCurves Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

CreateNewCurve Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : CreateNewCurve Method (ISurface)

Creates a new empty curve for the part.

.NET Syntax

Visual Basic (Declaration)

```
Function CreateNewCurve() As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.CreateNewCurve()
```

C#

```
System.object CreateNewCurve()
```

C++/CLI

```
System.Object^ CreateNewCurve();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Newly created [curve](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::CreateNewCurve](#).

Remarks

The intention is that with a handle on such (initially empty) curves, appropriate construction can be called that eventually amounts to non-trivial objects.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::ICreateNewCurve Method](#)
[ISurface::MakeIsoCurve Method](#)
[ISurface::MakeIsoCurves Method](#)
[ISurface::IMakeIsoCurve Method](#)
[ISurface::IMakeIsoCurves Method](#)
[ICurve::CreateTrimmedCurve2 Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

CreateTrimmedSheet Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : CreateTrimmedSheet Method (ISurface)

Obsolete. Superseded by [ISurface::CreateTrimmedSheet4](#).

.NET Syntax

Visual Basic (Declaration)

```
Function CreateTrimmedSheet( _  
    ByVal Curves As System.Object _  
) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim Curves As System.Object  
Dim value As System.Object  
  
value = instance.CreateTrimmedSheet(Curves)
```

C#

```
System.object CreateTrimmedSheet(  
    System.object Curves  
)
```

C++/CLI

```
System.Object^ CreateTrimmedSheet(  
& System.Object^ Curves  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Curves
Array of [curves](#) that represent the boundary of the sheet

Return Value

Temporary sheet [body](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::CreateTrimmedSheet](#).

Example

[Create Temporary Extruded Body \(C#\)](#)
[Create Temporary Extruded Body \(VB.NET\)](#)
[Create Temporary Extruded Body \(VBA\)](#)

Remarks

The array of curves represents all of the curves required to add the appropriate trimming loops to the surface. An empty entry in the array represents the separation between loops.

The curves supplied are assumed to lie on the surface. If the curves are 2D curves, then they should be created using this surface.

If your application is creating a trimmed sheet body from a periodic surface without trimming curves, then the curve array must contain one entry: null or Nothing.

If your application uses `ISurface::CreateTrimmedSheet`, then your application must also use [ICurve::CreateTrimmedCurve2](#) for the curves created by [IModeler::CreateArc](#) or [IModeler::ICreateArc](#) and [IModeler::CreateLine](#) or [IModeler::ICreateLine](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)

Availability

SOLIDWORKS 99, datecode 1999207

SOLIDWORKS API Help

[Send comments on this topic.](#)

CreateTrimmedSheet4 Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : CreateTrimmedSheet4 Method (ISurface)

Obsolete. Superseded by [ISurface::CreateTrimmedSheet5](#).

.NET Syntax

Visual Basic (Declaration)

```
Function CreateTrimmedSheet4(  
    ByVal Curves As System.Object, _  
    ByVal PreserveAnalyticCurves As System.Boolean _  
) As System.Object
```

Visual Basic (Usage)

```
Dim instance As ISurface  
Dim Curves As System.Object  
Dim PreserveAnalyticCurves As System.Boolean  
Dim value As System.Object  
  
value = instance.CreateTrimmedSheet4(Curves, PreserveAnalyticCurves)
```

C#



```
System.Object CreateTrimmedSheet4(  
    System.Object Curves,  
    System.bool PreserveAnalyticCurves  
)
```

C++/CLI

```
System.Object^ CreateTrimmedSheet4(  
& System.Object^ Curves,  
& System.bool PreserveAnalyticCurves  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Curves

Array of [curves](#) that represent the boundary of the sheet

PreserveAnalyticCurves

True to preserve analytic curves, false to store all trimming curves as SP-curves

Return Value

Temporary sheet [body](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::CreateTrimmedSheet4](#).

Example

[Create Temporary Elliptical Extrusion \(VBA\)](#)
[Create Temporary Elliptical Extrusion \(VB.NET\)](#)
[Create Temporary Elliptical Extrusion \(C#\)](#)
[Fill Holes in Part \(C#\)](#)
[Fill Holes in Part \(VB.NET\)](#)
[Fill Holes in Part \(VBA\)](#)

Remarks

The array of curves represents all of the curves required to add the appropriate trimming loops to the surface. A null entry in the array represents the separation between loops.

The curves supplied are assumed to lie on the surface. If the curves are 2D curves, then they should be created using this surface.

If your application is creating a trimmed sheet body from a periodic surface without trimming curves, then the curve array must contain one entry: null or Nothing.

Before calling this method, trim the curves created by [IModeler::CreateArc](#) and [IModeler::CreateLine](#) by calling [ICurve::CreateTrimmedCurve2](#). Elliptical curves created by [IModeler::CreateEllipse](#) do not need to be trimmed before calling this method.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::ICreateTrimmedSheet4 Method](#)

Availability

SOLIDWORKS 2007 SP4, Revision Number 15.4

SOLIDWORKS API Help

[Send comments on this topic.](#)

CreateTrimmedSheet5 Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : CreateTrimmedSheet5 Method (ISurface)

Creates a trimmed sheet body from this surface.

.NET Syntax

Visual Basic (Declaration)

```
Function CreateTrimmedSheet5( _  
    ByVal Curves As System.Object, _  
    ByVal PreserveAnalyticCurves As System.Boolean, _
```

```
ByVal Tolerance As System.Double _  
) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim Curves As System.Object  
Dim PreserveAnalyticCurves As System.Boolean  
Dim Tolerance As System.Double  
Dim value As System.Object  
  
value = instance.CreateTrimmedSheet5(Curves, PreserveAnalyticCurves, Tolerance)
```

C#

```
System.object CreateTrimmedSheet5(  
    System.object Curves,  
    System.bool PreserveAnalyticCurves,  
    System.double Tolerance  
)
```

C++/CLI

```
System.Object^ CreateTrimmedSheet5(  
& System.Object^ Curves,  
& System.bool PreserveAnalyticCurves,  
& System.double Tolerance  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Curves

Array of [curves](#) that represent the boundary of the trimmed sheet (see **Remarks**)

PreserveAnalyticCurves

True to preserve analytic curves, false to store all trimming curves as SP-curves

Tolerance

Tolerance for gaps between edges (see **Remarks**)

Return Value

Temporary sheet [body](#).

Visual Basic for Applications (VBA) Syntax

See [Surface::CreateTrimmedSheet5](#).

Example

[Create Spherical Surface, Trimmed Sheet, and Feature from Body \(VBA\)](#)

Remarks

The Curves array contains all of the curves required to add the appropriate trimming loops to the surface. A null or Nothing entry in the array represents the separation between loops.

The trimming curves supplied are assumed to lie on the surface. If the curves are 2D curves, then they should be created using this surface.

If your application is creating a trimmed sheet body from a periodic surface without trimming curves, then the Curves array should contain only one entry: null or Nothing.

Tolerance sets the edge precision in PK_SURF_make_sheet_trimmed. The default parasolid tolerance is 0.00001. This parameter allows you to specify a looser tolerance for gaps between edges.

Before calling this method, trim the curves created by [IModeler::CreateArc](#) and [IModeler::CreateLine](#) by calling [ICurve::CreateTrimmedCurve2](#). Elliptical curves created by [IModeler::CreateEllipse](#) do not need to be trimmed before calling this method.

See Also

[ISurface Interface](#)
[ISurface Members](#)

Availability

CylinderParams Property (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : CylinderParams Property (ISurface)

Gets the parameters of a cylindrical surface.

Visual Basic (.NET) Syntax

Visual Basic (Declaration)

```
ReadOnly Property CylinderParams As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface
Dim value As System.Object

value = instance.CylinderParams
```

C#

```
System.Object CylinderParams {get;}
```

C++/CLI

```
property System.Object^ CylinderParams {
    System.Object^ get();
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

Array of doubles describing the parameters of a cylindrical surface

Visual Basic for Applications (VBA) Syntax

See [Surface::CylinderParams](#).

Example

[Get Angle of Hole Not Normal to a Face \(VBA\)](#)
[Get Entities Attached to Cosmetic Thread \(VBA\)](#)
[Get Parameters of Cylindrical Surface \(VBA\)](#)

Remarks

Returns an array of 7 doubles:

- *origin.x*
- *origin.y*
- *origin.z*
- *axis.x*
- *axis.y*
- *axis.z*
- *radius*

The *origin* and *radius* parameters are in meters.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::CylinderParams Property](#)
[ISurface::ISCylinder Method](#)

Evaluate Method (ISurface)

[See Also](#) [Example](#)



Evaluates the surface, given the u and v parameters of the surface.

.NET Syntax

Visual Basic (Declaration)

```
Function Evaluate( _  
    ByVal UParam As System.Double, _  
    ByVal VParam As System.Double, _  
    ByVal NumUDeriv As System.Integer, _  
    ByVal NumVDeriv As System.Integer _  
    ) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim UParam As System.Double  
Dim VParam As System.Double  
Dim NumUDeriv As System.Integer  
Dim NumVDeriv As System.Integer  
Dim value As System.Object  
  
value = instance.Evaluate(UParam, VParam, NumUDeriv, NumVDeriv)
```

C#

```
System.object Evaluate(  
    System.double UParam,  
    System.double VParam,  
    System.int NumUDeriv,  
    System.int NumVDeriv  
)
```

C++/CLI

```
System.Object^ Evaluate(  
    & System.double UParam,  
    & System.double VParam,  
    & System.int NumUDeriv,  
    & System.int NumVDeriv  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

UParam

Value of u parameter

VParam

Value of v parameter

NumUDeriv

Number of u derivatives required

NumVDeriv

Number of v derivatives required

Return Value

This method returns the evaluated point, normal, and derivatives with respect to u and v up to order NumUDerivs and NumVDerivs, respectively. If NumUDerivs and NumVDerivs are 0, then no derivatives are returned.

For more information on the valid u and v parameters of the surface, see [ISurface::Parameterization](#) or [ISurface::IParameterization](#). Also, use [IFace2::FaceInSurfaceSense](#) to check to see if the normal of the face is in the same direction or in the opposite direction of the surface normal.

The evaluation returns the following array of (3 * (((NumUDerivs + 1) * (NumVDerivs + 1)) + 1)) doubles:

$$\left[\text{evaluatedPoint}[3], \text{evaluatedDerivatives}[xx], \text{evaluatedNormal}[3] \right]$$

where:

$\text{evaluatedPoint}[3]$ = point representing the evaluated X,Y,Z point in meters

$\text{evaluatedDerivatives}[xx]$ = array of vectors representing the derivatives

where the n th derivative wrt u ($i \leq \text{NumUDerivs}$)

and the n th derivative wrt v ($j \leq \text{NumVDerivs}$)

would be vector number $(i + (\text{NumUDerivs} + 1) * j) - 1$ in the *evaluatedDerivatives* array.

If *NumUDerivs* and *NumVDerivs* are 0, then no derivatives are returned.

evaluatedNormal[3] = a vector representing the evaluated normal

The following table describes the number of derivatives that can be requested based on the surface type. If you request more derivatives than allowed, then this method returns all data as zeros. To determine the surface type, see [ISurface::Identity](#).

U Derivatives				
		0	1	2
	0	a	a	b
V Derivatives	1	a	a	c
	2	b	c	d

where:

- a - All surface types
- b - All surface types except blend surfaces
- c - All surface types except blend surfaces and offset surfaces
- d - All surface types except blend surfaces but offset surfaces will only return 0th, 1st, and 2nd derivatives (the uncalculated 3rd and 4th derivatives are returned as 0 vectors).

For example, if you specified 2 derivatives in both u and v, then an array containing 10 vectors is returned. The returned data is in model space coordinates.

```
[P(u,v)      P(u,v)/du      P(u,v)/dudu]
[P(u,v)/dv    P(u,v)/dudv    P(u,v)/dududv]
[P(u,v)/dv dv P(u,v)/dudv dv P(u,v)/dududv dv]
[N(u,v)]
```

where:

- du the partial derivative in the u direction
- dv the partial derivative in the v direction
- N(u,v) the normal vector
- Each P() represents a model space point or vector depending on the entry in the array

If you request 2 derivatives for u and 1 for v, then an array containing 7 vectors is returned:

```
[P(u,v)      P(u,v)/du      P(u,v)/dudu]
[P(u,v)/dv    P(u,v)/dudv    P(u,v)/dududv]
[N(u,v)]
```

If u = 1 and v = 2, then:

```
[P(u,v)      P(u,v)/du]
[P(u,v)/dv    P(u,v)/dudv]
[P(u,v)/dv dv P(u,v)/dudv dv]
[N(u,v)]
```

Visual Basic for Applications (VBA) Syntax

See [Surface::Evaluate](#).

Example

[Create Trimmed Curve \(VBA\)](#)
[Create Trimmed Curve \(VB.NET\)](#)
[Create Trimmed Curve \(C#\)](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IEvaluate Method](#)
[ISurface::IEvaluateAtPoint Method](#)
[ISurface::EvaluateAtPoint Method](#)

EvaluateAtPoint Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : EvaluateAtPoint Method (ISurface)

Evaluates a surface at the specified XYZ point.

Visual Basic Syntax

Visual Basic (Declaration)

```
Function EvaluateAtPoint( _  
    ByVal PositionX As System.Double, _  
    ByVal PositionY As System.Double, _  
    ByVal PositionZ As System.Double _  
) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim PositionX As System.Double  
Dim PositionY As System.Double  
Dim PositionZ As System.Double  
Dim value As System.Object  
  
value = instance.EvaluateAtPoint(PositionX, PositionY, PositionZ)
```

C#

```
System.object EvaluateAtPoint(  
    System.double PositionX,  
    System.double PositionY,  
    System.double PositionZ  
)
```

C++/CLI

```
System.Object^ EvaluateAtPoint(  
& System.double PositionX,  
& System.double PositionY,  
& System.double PositionZ  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

PositionX
X position

PositionY
Y position

PositionZ
Z position

Return Value

Array of doubles (see **Remarks**)

Visual Basic for Applications (VBA) Syntax

See [Surface.EvaluateAtPoint](#).

Example

[Determine Type of Face \(VBA\)](#)
[Evaluate Points on Surface \(VBA\)](#)
[Select Tangent Faces \(VBA\)](#)
[Select Edges of All Holes on Face \(C#\)](#)
[Select Edges of All Holes on Face \(VB.NET\)](#)
[Select Edges of All Holes on Face \(VBA\)](#)

Remarks

This method calculates the normal, the principal directions, and the principal curvatures, of the surface at the specified point.

Use [IFace2::FaceInSurfaceSense](#) to check the directions of the face normal and surface normal. [IFace2::FaceInSurfaceSense](#) returns true when the face normal and surface normal point in opposite directions, and false when they point in the same direction.

The return value is the following array of eleven doubles:

`[surfNorm[i, j, k], principalDir1[i, j, k], principalDir2[i, j, k], principalCurvature1, principalCurvature2]`

where:

`surfNorm[i, j, k]` = normalized vector describing the surface normal

`principalDir1[i, j, k]` = normalized vector describing the first principal direction

`principalDir2[i, j, k]` = normalized vector describing the second principal direction

`principalCurvature1` = first principal curvature

`principalCurvature2` = second principal curvature

Principal Curvature 1 is the minimum normal curvature at the point (largest radius). Principal Curvature 2 is the maximum normal curvature at the point.

The tangent direction producing Principal Curvature 1 is called the first principal direction, and the tangent direction producing Principal Curvature 2 is called the second principal direction.

It is a property of differentiable surfaces that `principalDir1` and `principalDir2` are orthogonal.

A positive curvature by convention implies a centre of curvature on the side pointed away from by the surface normal (convex).

See "Faux and Pratt Computational Geometry for Design and Manufacture" for more information.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::EvaluateAtPoint Method](#)
[ISurface::Evaluate Method](#)
[ISurface::Evaluate Method](#)
[ISurface::Parameterization Method](#)
[ISurface::ReverseEvaluate Method](#)
[ISurface::Parameterization Method](#)
[ISurface::ReverseEvaluate Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

FindMinimumRadius Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : FindMinimumRadius Method (ISurface)

Gets the minimum radius of curvature for the selected surface.

.NET Syntax

Visual Basic (Declaration)

```
Function FindMinimumRadius( _  
    ByVal UBound As System.Object, _  
    ByVal VBound As System.Object, _  
    ByRef NumOfRadius As System.Integer, _  
    ByRef Radius As System.Object, _  
    ByRef Location As System.Object, _  
    ByRef UVParameter As System.Object _  
    ) As System.Boolean
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim UBound As System.Object  
Dim VBound As System.Object  
Dim NumOfRadius As System.Integer  
Dim Radius As System.Object  
Dim Location As System.Object  
Dim UVParameter As System.Object  
Dim value As System.Boolean  
  
value = instance.FindMinimumRadius(UBound, VBound, NumOfRadius, Radius, Location, UVParameter)
```

C#

```
System.bool FindMinimumRadius(  
    System.object UBound,
```

```

System.object VBound,
out System.int NumOfRadius,
out System.object Radius,
out System.object Location,
out System.object UVParameter
)

```

C++/CLI

```

System.bool FindMinimumRadius(
& System.Object^ UBound,
& System.Object^ VBound,
& [Out] System.int NumOfRadius,
& [Out] System.Object^ Radius,
& [Out] System.Object^ Location,
& [Out] System.Object^ UVParameter
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

UBound
MinMax UParameter
VBound
MinMax VParameter
NumOfRadius
Number of radius; can be 0, 1, or 2
Radius
Minimum radius of curvature (see **Remarks**)
Location
[Position](#) where the minimum radius curvature occurred (see **Remarks**)
UVParameter
[UV parameters](#); because points are UV, third ordinates are 0 (see **Remarks**)

Return Value

True if operation succeeds, false if it fails

Visual Basic for Applications (VBA) Syntax

See [Surface::FindMinimumRadius](#).

Example

[Find Minimum Radius Curvature of Surface \(VBA\)](#)

Remarks

The search is confined to the portion of the selected curve lying inside of UBound and VBound.

COM returns these data types for these parameters:

- Radius: VARIANT of SafeDoubleArray
- Location: VARIANT of SafeDispatchArray of [IMathpoint](#)
- UVParameter: VARIANT of SafeDispatchArray of [IMathpoint](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::FindMinimumRadius Method](#)
[ISurface::Parameterization Method](#)
[ISurface::Parameterization Method](#)
[ICurve::FindMinimumRadius Method](#)
[ICurve::FindMinimumRadius Method](#)
[IFace2::GetUVBounds Method](#)
[IFace2::GetUVBounds Method](#)

Availability

SOLIDWORKS 2009 FCS, Revision Number 17.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetBSurfParams Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetBSurfParams Method (ISurface)

Obsolete. Superseded by [ISurface::GetBSurfParams2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function GetBSurfParams(  
    ByVal WantCubicRational As System.Boolean, _  
    ByVal VP0 As System.Object _  
) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim WantCubicRational As System.Boolean  
Dim VP0 As System.Object  
Dim value As System.Object  
  
value = instance.GetBSurfParams(WantCubicRational, VP0)
```

C#

```
System.object GetBSurfParams(  
    System.bool WantCubicRational,  
    System.object VP0  
)
```

C++/CLI

```
System.Object^ GetBSurfParams(  
& System.bool WantCubicRational,  
& System.Object^ VP0  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

WantCubicRational

VP0

Visual Basic for Applications (VBA) Syntax

See [Surface::GetBSurfParams](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetBSurfParams2 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetBSurfParams2 Method (ISurface)

Obsolete. Superseded by [ISurface::GetBSurfParams3](#).

.NET Syntax

Visual Basic (Declaration)

```
Function GetBSurfParams2( _  
    ByVal WantCubic As System.Boolean, _  
    ByVal WantNonRational As System.Boolean, _  
    ByVal VP0 As System.Object, _  
    ByVal Tolerance As System.Double, _  
    ByRef Sense As System.Boolean _  
) As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim WantCubic As System.Boolean
```

```

Dim WantNonRational As System.Boolean
Dim VP0 As System.Object
Dim Tolerance As System.Double
Dim Sense As System.Boolean
Dim value As System.Object

value = instance.GetBSurfParams2(WantCubic, WantNonRational, VP0, Tolerance, Sense)

```

C#

```

System.object GetBSurfParams2(
    System.bool WantCubic,
    System.bool WantNonRational,
    System.object VP0,
    System.double Tolerance,
    out System.bool Sense
)

```

C++/CLI

```

System.Object^ GetBSurfParams2(
& System.bool WantCubic,
& System.bool WantNonRational,
& System.Object^ VP0,
& System.double Tolerance,
& [Out] System.bool Sense
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

WantCubic

True if cubic is needed, false if not; specifying true converts any surface type to a cubic Bsurface

WantNonRational

True if non-rational is needed, false if not; specifying true converts any surface type to a non-rational Bsurface; if you specify true, then you should only use this method for surfaces that are Bsurface or blend surface; otherwise, the underlying call is not made and the values returned from this are not initialized or contain the values from the last call

VP0

Array of values describing the UV range of the surface to be output; you can obtain these values by using [ISurface::Parameterization](#) or [ISurface::IParameterization](#)

Tolerance

Tolerance, in meters, between the approximated b-spline surface and the underlying surface; the default value is 0.01 and should generally be reduced to the tolerance desired

Sense

Approximated b-spline surface is not always in the same direction as the original surface; if sense is true, then the underlying surface and the b-spline surface are in the same direction

Return Value

Array of values giving the b-spline surface parameters

Visual Basic for Applications (VBA) Syntax

See [Surface::GetBSurfParams2](#).

Remarks

The VP0 parameter contains the following values, which you can obtain by using [ISurface::Parameterization](#) or [ISurface::IParameterization](#):

- vP0[0] & vP0[2] are the lower bounds of the U & V surface parameters, respectively
- vP0[1] & vP0[3] are the upper bounds of the U & V surface parameters, respectively

If you want to use the Bsurface in combination with its trim curves, you should use the [IFace2::GetTrimCurves2](#) method to extract the trim curves and the Bsurface. The IFace2::GetTrimCurves2 method provides much better alignment of the trim curves with the Bsurface because they are both generated at the same time.

Evaluation is as follows:

Number of returned double elements is (4 + NumUKnots + NumVKnots + NumControlPointDoubles), where NumUKnots = (NumColumnsControlPoint + Uorder) and NumVKnots = (NumRowsControlPoints + Vorder).

The returned data are arranged in this order:

- Uorder, Vorder - These are two ints packed into a double
 - Uorder - Order of surface in U direction
 - Vorder - Order of surface in V direction
- NumColumnsControlPoints, NumRowsControlPoints - These are two integers packed into a double
 - NumColumnsControlPoints - Number of columns for the control points

- NumRowsControlPoints - Number of rows for the control points
- Uperiodicity, Vperiodicity - These are two integers packed into a double. Since you may be generating a Bsurface with this function, the U,V periodicity values may be opposite from the original surface. Refer to ISurface::Parameterization or ISurface::IParameterization.
 - Uperiodicity - True if surface is periodic in U direction
 - Vperiodicity - True if surface is periodic in V direction
- DimensionControlPoints, dummy - These are two integers packed into a double
 - DimensionControlPoints - Dimension for control points.
 - DimensionControlPoints = 3 for non-rational surfaces and control point coordinates are output as [x0,y0,z0,x1,y1,z1,.....].
 - DimensionControlPoints = 4 for rational surfaces and control point coordinates and the weight are output as [x0,y0,z0,w0,x1,y1,z1,w1,.....].
 - dummy - a system-used filler.
- UKnots - Knot vector in the u direction.
There will be (NumColumnsControlPoints + Uorder) knot values. If the surface is periodic in the U direction, then data is converted and returned in a non-periodic form with additional knots added to the surface ends.
- VKnots - Knot vector in the v direction.
There will be (NumRowsControlPoints + Vorder) knot values. If the surface is periodic in the V direction, then data is converted and returned in a non-periodic form with additional knots added to the surface ends.
- ControlPoints - This is a list of control points output in (NumColumnsControlPoints*NumRowsControlPoints) vectors of dimension DimensionControlPoints. The vectors are output row by row. The U direction of surface is in the row direction. The V direction of surface is in the column direction.
 - For non-rational surfaces, control point coordinates are output as [x0,y0,z0,x1,y1,z1,.....].
 - For rational surfaces, control point coordinates and the weight are output as [x0,y0,z0,w0,x1,y1,z1,w1,.....].
 - For example:

Row1, Column1	Row1, Column2	
x0, y0, z0	x1, y1, z1	= 6
Row2, Column1	Row2, Column2	
x2, y2, z2	x3, y3, z3	= 6
Row3, Column1	Row3, Column2	
x4, y4, z4	x5, y5, z5	= 6
		= 18

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IGetBSurfParams Method](#)
[ISurface::IGetBSurfParamsSize3 Method](#)

Availability

SOLIDWORKS 2000 SP3, Revision Number 8.3

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetBSurfParams3 Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetBSurfParams3 Method (ISurface)

Gets the parameterization data for a B-spline surface.

.NET Syntax

Visual Basic (Declaration)

```
Function GetBSurfParams3( _  
    ByVal WantCubic As System.Boolean, _  
    ByVal WantNonRational As System.Boolean, _  
    ByVal VP0 As System.Object, _  
    ByVal Tolerance As System.Double, _  
    ByRef Sense As System.Boolean _  
) As BSurfParamData
```

Visual Basic (Usage)

```
Dim instance As ISurface  
Dim WantCubic As System.Boolean
```



```

Dim WantNonRational As System.Boolean
Dim VP0 As System.Object
Dim Tolerance As System.Double
Dim Sense As System.Boolean
Dim value As BSurfParamData

value = instance.GetBSurfParams3(WantCubic, WantNonRational, VP0, Tolerance, Sense)

```

C#

```

BSurfParamData GetBSurfParams3(
    System.bool WantCubic,
    System.bool WantNonRational,
    System.object VP0,
    System.double Tolerance,
    out System.bool Sense
)

```

C++/CLI

```

BSurfParamData^ GetBSurfParams3(
    & System.bool WantCubic,
    & System.bool WantNonRational,
    & System.Object^ VP0,
    & System.double Tolerance,
    [Out] System.bool Sense
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

WantCubic

True if cubic is needed, false if not; specifying true converts any surface to a cubic B-spline surface

WantNonRational

True if non-rational is needed, false if not; specifying true converts any surface to a non-rational B-spline surface; if you specify true, then you should only use this method for surfaces that are of B-spline or blend type; otherwise, the underlying call is not made and the values returned from this are not initialized, or they contain the values from the last call

VP0

[ISurfaceParameterizationData](#)

Tolerance

Tolerance, in meters, between the approximated B-spline surface and the underlying surface; the default value is 0.01 and should generally be reduced to the tolerance desired

Sense

Approximated B-spline surface is not always in the same direction as the original surface; if Sense is true, then the underlying surface and the B-spline surface are in the same direction

Return Value

An [IBSurfParamData](#) object

Visual Basic for Applications (VBA) Syntax

See [Surface::GetBSurfParams3](#).

Example

[Get B-Spline Surface Parameterization Data \(C#\)](#)

[Get B-Spline Surface Parameterization Data \(VB.NET\)](#)

[Get B-Spline Surface Parameterization Data \(VBA\)](#)

Remarks

Before calling this method, call [ISurface.Parameterization2](#) to populate VP0.

If you want to use the B-spline surface in combination with its trim curves, you should use the [IFace2::GetTrimCurves2](#) method to extract both the trim curves and the B-spline surface. The [IFace2::GetTrimCurves2](#) method provides much better alignment of the trim curves with the B-spline surface, because they are both generated at the same time.

See Also

[ISurface Interface](#)

[ISurface Members](#)

[ISurface::IGetBSurfParams Method](#)

[ISurface::IGetBSurfParamsSize3 Method](#)

Availability

GetClosestPointOn Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetClosestPointOn Method (ISurface)

Uses the X,Y,Z input point to determine the closest point on the surface.

.NET Syntax

Visual Basic (Declaration)

```

Function GetClosestPointOn( _
    ByVal X As System.Double, _
    ByVal Y As System.Double, _
    ByVal Z As System.Double _
) As System.Object

```

Visual Basic (Usage)



```

Dim instance As ISurface
Dim X As System.Double
Dim Y As System.Double
Dim Z As System.Double
Dim value As System.Object

value = instance.GetClosestPointOn(X, Y, Z)

```

C#

```

System.object GetClosestPointOn(
    System.double X,
    System.double Y,
    System.double Z
)

```

C++/CLI

```

System.Object^ GetClosestPointOn(
    & System.double X,
    & System.double Y,
    & System.double Z
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

X
x coordinate of the approximate position on the surface

Y
y coordinate of the approximate position on the surface

Z
z coordinate of the approximate position on the surface

Return Value

Array of 5 doubles containing the (x,y,z) coordinates and the (u,v) parameters of the point on the surface

Visual Basic for Applications (VBA) Syntax

See [Surface::GetClosestPointOn](#).

Example

[Create Imported Surface Body from Sketch \(C#\)](#)
[Cut Body in Half Using Macro Feature \(VBA\)](#)
[Evaluate Points on Surface \(VBA\)](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)

[ISurface::IGetClosestPointOn Method](#)
[ISurface::GetProjectedPointOn Method](#)

Availability

SOLIDWORKS 2001 FCS, Revision Number 9.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetExtrusionsurfParams Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetExtrusionsurfParams Method (ISurface)

Gets the parameters for the extrusion surface.

.NET Syntax

Visual Basic (Declaration)

```
Function GetExtrusionsurfParams() As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.GetExtrusionsurfParams()
```

C#

```
System.object GetExtrusionsurfParams()
```

C++/CLI

```
System.Object^ GetExtrusionsurfParams();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Array of doubles giving extrusion surface parameters

Visual Basic for Applications (VBA) Syntax

See [Surface::GetExtrusionsurfParams](#).

Remarks

An extrusion surface is constructed by extruding a profile curve (line or arc or b-spline) along a direction vector. You can retrieve the profile curve for the extrusion using [ISurface::GetProfileCurve](#) or [ISurface::IGetProfileCurve](#).

Three doubles of parameters returned, *DirectionVector[3]*, which is the direction vector along which the profile curve is extruded.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IGetExtrusionsurfParams Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetIntersectCurveCount Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetIntersectCurveCount Method (ISurface)

Obsolete. Superseded by [ISurface::GetIntersectCurveCount2](#).

.NET Syntax

Visual Basic (Declaration)

```
Function GetIntersectCurveCount( _  
    ByVal OtherCurve As Curve, _
```

```
ByRef CurveBound As System.Double _  
) As System.Integer
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim OtherCurve As Curve  
Dim CurveBound As System.Double  
Dim value As System.Integer  
  
value = instance.GetIntersectCurveCount(OtherCurve, CurveBound)
```

C#

```
System.int GetIntersectCurveCount(  
    Curve OtherCurve,  
    ref System.double CurveBound  
)
```

C++/CLI

```
System.int GetIntersectCurveCount(  
& Curve^ OtherCurve,  
& System.double% CurveBound  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

OtherCurve

CurveBound

Visual Basic for Applications (VBA) Syntax

See [Surface::GetIntersectCurveCount](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetIntersectCurveCount2 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetIntersectCurveCount2 Method (ISurface)

Gets the number of points for a surface-curve intersection.

.NET Syntax

Visual Basic (Declaration)

```
Function GetIntersectCurveCount2( _  
    ByVal OtherCurve As Curve, _  
    ByRef CurveBound As System.Double _  
) As System.Integer
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim OtherCurve As Curve  
Dim CurveBound As System.Double  
Dim value As System.Integer  
  
value = instance.GetIntersectCurveCount2(OtherCurve, CurveBound)
```

C#

```
System.int GetIntersectCurveCount2(  
    Curve OtherCurve,
```

```
    ref System.double CurveBound
)
```

C++/CLI

```
System.int GetIntersectCurveCount2(
&    Curve^ OtherCurve,
&    System.double% CurveBound
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

OtherCurve

[Curve](#)

CurveBound

Array of 6 doubles representing the start and end points of the curve

Return Value

Number of points

Visual Basic for Applications (VBA) Syntax

See [Surface::GetIntersectCurveCount2](#).

Remarks

Visual Basic and C++ Dispatch applications should use [ISurface::IntersectCurve2](#) instead of this method.

See Also

[ISurface Interface](#)

[ISurface Members](#)

[ISurface::IntersectCurve2 Method](#)

Availability

SOLIDWORKS 2006 FCS, Revision Number 14.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetIntersectSurfaceCount Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetIntersectSurfaceCount Method (ISurface)

Gets the number of curves for a surface-surface intersection.

.NET Syntax

Visual Basic (Declaration)

```
Function GetIntersectSurfaceCount( _
    ByVal OtherSurface As Surface _
) As System.Integer
```

Visual Basic (Usage)



```
Dim instance As ISurface
Dim OtherSurface As Surface
Dim value As System.Integer

value = instance.GetIntersectSurfaceCount(OtherSurface)
```

C#

```
System.int GetIntersectSurfaceCount(
    Surface OtherSurface
)
```

C++/CLI

```
System.int GetIntersectSurfaceCount(
&    Surface^ OtherSurface
```

)

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

OtherSurface
Other [surface](#)

Return Value

Number of curves

 Visual Basic for Applications (VBA) Syntax

See [Surface::GetIntersectSurfaceCount](#).

 See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IntersectSurface Method](#)
[ISurface::IntersectSurface Method](#)

 Availability

SOLIDWORKS 2001Plus FCS, Revision Number 10.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetOffsetSurfParams Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetOffsetSurfParams Method (ISurface)

Obsolete. Superseded by [ISurface::GetOffsetSurfParams2](#).

 .NET Syntax

Visual Basic (Declaration)

```
Function GetOffsetSurfParams() As System.Double
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Double  
  
value = instance.GetOffsetSurfParams()
```

C#

```
System.double GetOffsetSurfParams()
```

C++/CLI

```
System.double GetOffsetSurfParams();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

 Visual Basic for Applications (VBA) Syntax

See [Surface::GetOffsetSurfParams](#).

 See Also

[ISurface Interface](#)
[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetOffsetSurfParams2 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetOffsetSurfParams2 Method (ISurface)

Gets the overall offset distance of this offset surface.

Visual Basic Syntax

Visual Basic (Declaration)

```
Function GetOffsetSurfParams2( _  
    ByRef BaseSurf As System.Object, _  
    ByRef Sense As System.Boolean _  
) As System.Double
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim BaseSurf As System.Object  
Dim Sense As System.Boolean  
Dim value As System.Double  
  
value = instance.GetOffsetSurfParams2(BaseSurf, Sense)
```

C#

```
System.double GetOffsetSurfParams2(  
    out System.object BaseSurf,  
    out System.bool Sense  
)
```

C++/CLI

```
System.double GetOffsetSurfParams2(  
& [Out] System.Object^ BaseSurf,  
& [Out] System.bool Sense  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

BaseSurf

Base [surface](#) used to generate the offset surface

Sense

If true, then the offset is in the direction of the original surface's normal, if false, then the offset is in the opposite direction of the original surface's normal

Return Value

Offset for the surface

Visual Basic for Applications (VBA) Syntax

See [Surface::GetOffsetSurfParams2](#).

Remarks

This method can get a pointer to the base surface and its sense. Both BaseSurf and Sense parameters can be a NULL pointer.

See Also

[ISurface Interface](#)

[ISurface Members](#)

[ISurface::GetOffsetSurfParams2 Method](#)

Availability

SOLIDWORKS 2001 FCS, Revision Number 9.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetProfileCurve Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetProfileCurve Method (ISurface)

Gets the curve used to create the surface of revolution or extrusion surface and applies to only surface of revolution or extrusion surface.

Visual Basic Syntax

Visual Basic (Declaration)

```
Function GetProfileCurve() As System.Object
```

Visual Basic (Usage)

```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.GetProfileCurve()
```

C#

```
System.Object GetProfileCurve()
```

C++/CLI

```
System.Object^ GetProfileCurve();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Profile [curve](#) used to create the surface of revolution or extrusion surface

Visual Basic for Applications (VBA) Syntax

See [Surface::GetProfileCurve](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::GetProfileCurve Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetProjectedPointOn Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetProjectedPointOn Method (ISurface)

Gets the point where the input point is projected on to this surface.

.NET Syntax

Visual Basic (Declaration)

```
Function GetProjectedPointOn( _  
    ByVal Point As MathPoint, _  
    ByVal Direction As MathVector _  
) As MathPoint
```

Visual Basic (Usage)

```
Dim instance As ISurface  
Dim Point As MathPoint  
Dim Direction As MathVector  
Dim value As MathPoint  
  
value = instance.GetProjectedPointOn(Point, Direction)
```

C#

```
MathPoint GetProjectedPointOn(  
    MathPoint Point,  
    MathVector Direction  
)
```

C++/CLI

```
MathPoint^ GetProjectedPointOn(  
& MathPoint^ Point,  
& MathVector^ Direction  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

Point
[Point](#) to project
Direction
[Direction](#) to project the point

Return Value

[Point](#) where the input point is projected onto the surface

 Visual Basic for Applications (VBA) Syntax

See [Surface::GetProjectedPointOn](#).

 Example

[Get Projected Point \(VBA\)](#)

 See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::GetClosestPointOn Method](#)
[ISurface::IGetClosestPointOn Method](#)

 Availability

SOLIDWORKS 207 FCS, Revision Number 15.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

GetRevsurfParams Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : GetRevsurfParams Method (ISurface)

Gets the parameters for the surface.

 .NET Syntax

Visual Basic (Declaration)

```
Function GetRevsurfParams() As System.Object
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Object  
  
value = instance.GetRevsurfParams()
```

C#

```
System.object GetRevsurfParams()
```

C++/CLI

```
System.Object^ GetRevsurfParams();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Array of doubles (see **Remarks**)

 Visual Basic for Applications (VBA) Syntax

See [Surface::GetRevsurfParams](#).

 Remarks

A surface of revolution is constructed by revolving a profile curve (line or arc or b-spline) about an axis. The profile curve for the surface of revolution can be retrieved with [ISurface::GetProfileCurve](#) or [ISurface::IGetProfileCurve](#).

Eight doubles of parameters returned.

- *Point [3]* - X, Y, Z coordinates of the start point of the axis of rotation.
- *AxisDir[3]* - Vector showing the direction of the axis of rotation.
- *Params[2]* - Parameter range of the part of the profile curve that was used to create the surface.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::IGetRevsurfParams Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

IAddTrimmingLoop Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IAddTrimmingLoop Method (ISurface)

Obsolete. Superseded by [ISurface::IAddTrimmingLoop2](#).

.NET Syntax

Visual Basic (Declaration)

```
Sub IAddTrimmingLoop( _  
    ByVal CurveCount As System.Integer, _  
    ByRef Order As System.Integer, _  
    ByRef Dim As System.Integer, _  
    ByRef Periodic As System.Integer, _  
    ByRef NumKnots As System.Integer, _  
    ByRef NumCtrlPoints As System.Integer, _  
    ByRef Knots As System.Double, _  
    ByRef CtrlPointDbls As System.Double _  
)
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim CurveCount As System.Integer  
Dim Order As System.Integer  
Dim Dim As System.Integer  
Dim Periodic As System.Integer  
Dim NumKnots As System.Integer  
Dim NumCtrlPoints As System.Integer  
Dim Knots As System.Double  
Dim CtrlPointDbls As System.Double  
  
instance.IAddTrimmingLoop(CurveCount, Order, Dim, Periodic, NumKnots, NumCtrlPoints, Knots,  
    CtrlPointDbls)
```

C#

```
void IAddTrimmingLoop(  
    System.int CurveCount,  
    ref System.int Order,  
    ref System.int Dim,  
    ref System.int Periodic,  
    ref System.int NumKnots,  
    ref System.int NumCtrlPoints,  
    ref System.double Knots,  
    ref System.double CtrlPointDbls  
)
```

C++/CLI

```
void IAddTrimmingLoop(  
& System.int CurveCount,  
& System.int% Order,  
& System.int% Dim,  
& System.int% Periodic,  
& System.int% NumKnots,  
& System.int% NumCtrlPoints,  
& System.double% Knots,
```

```
& System.double% CtrlPointDbls
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

CurveCount

Order

Dim

Periodic

NumKnots

NumCtrlPoints

Knots

CtrlPointDbls

Visual Basic for Applications (VBA) Syntax

See [Surface::IAddTrimmingLoop](#).

See Also

[ISurface Interface](#)

[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

IAddTrimmingLoop2 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IAddTrimmingLoop2 Method (ISurface)

Creates a trimming loop out of specified surface parametric (UV-curves) and adds it to a list of such loops.

.NET Syntax

Visual Basic (Declaration)

```
Sub IAddTrimmingLoop2( _
    ByVal CurveCount As System.Integer, _
    ByRef Order As System.Integer, _
    ByRef Dim As System.Integer, _
    ByRef Periodic As System.Integer, _
    ByRef NumKnots As System.Integer, _
    ByRef NumCtrlPoints As System.Integer, _
    ByRef Knots As System.Double, _
    ByRef CtrlPointDbls As System.Double, _
    ByRef UvRange As System.Double _
)
```

Visual Basic (Usage)



```
Dim instance As ISurface
Dim CurveCount As System.Integer
Dim Order As System.Integer
Dim Dim As System.Integer
Dim Periodic As System.Integer
Dim NumKnots As System.Integer
Dim NumCtrlPoints As System.Integer
Dim Knots As System.Double
Dim CtrlPointDbls As System.Double
Dim UvRange As System.Double
```

```
instance.IAddTrimmingLoop2(CurveCount, Order, Dim, Periodic, NumKnots, NumCtrlPoints, Knots,
CtrlPointDbls, UvRange)
```

C#

```
void IAddTrimmingLoop2(
    System.int CurveCount,
    ref System.int Order,
    ref System.int Dim,
```

```

    ref System.int Periodic,
    ref System.int NumKnots,
    ref System.int NumCtrlPoints,
    ref System.double Knots,
    ref System.double CtrlPointDbls,
    ref System.double UvRange
)

```

C++/CLI

```

void IAddTrimmingLoop2(
& System.int CurveCount,
& System.int% Order,
& System.int% Dim,
& System.int% Periodic,
& System.int% NumKnots,
& System.int% NumCtrlPoints,
& System.double% Knots,
& System.double% CtrlPointDbls,
& System.double% UvRange
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

CurveCount

Number of surface parametric (UV) curves constituting the loop; the size of Order, Dim, Periodic, NumKnots, and NumControlPnts arrays

Order

Orders of the curves; array of CurveCount longs (VBA), Integers (VB.NET), or ints (C#, C++)

Dim

Dimensions of the curves' control points; array of CurveCount longs (VBA), Integers (VB.NET), or ints (C#, C++); if you set the first value in this array to negative of its absolute value, then 3D trim curves are expected

Periodic

0 for non-periodic or 1 for periodic; array of CurveCount longs (VBA), Integers (VB.NET), or ints (C#, C++)

NumKnots

Number of knots in the curves; array of CurveCount longs (VBA), Integers (VB.NET), or ints (C#, C++)

NumCtrlPoints

Number of control points in the curves; array of CurveCount longs (VBA), Integers (VB.NET), or ints (C#, C++)

Knots

Knot vectors of the curves; array of <TotalNumKnots> doubles, where TotalNumKnots = (TotalNumKnots + NumKnots[i]) for i = 1 to CurveCount

CtrlPointDbls

Control point coordinates of the curves; array of <TotalNumCPCoords> doubles, where TotalNumCPCoords = (TotalNumCPCoords + (Dim[i] * NumCtrlPoints[i])) for i = 1 to CurveCount

UvRange

Array of four doubles defining U Low U High V Low V High

Visual Basic for Applications (VBA) Syntax

See [Surface::IAddTrimmingLoop2](#).

Remarks

The list of trimming loops is created by a previous call to one of the base surface creation functions, [IBody2::CreateRevolutionSurface](#) or [IBody2::CreatePlanarSurface](#).

The information on each UV-curve is in b-spline form (knots and control point coordinates) and is compacted into two arrays, VKnots and VCtrlPointDbls.

Order of each curve in Order is, at a minimum, 2. A second order curve is a linear curve.

For each curve in NumCtrlPoints, Order, and NumKnots, (NumCtrlPoints >= Order) and (NumKnots = NumCtrlPoints + Order).

Dim for each curve in Dim specifies the dimension of each CtrlPointDbl set in CtrlPointDbls. This method expects 2D-polynomial (X,Y) or 2D-rational (X,Y,Weight) curves to be passed as trimming curves. However, you can set a flag so that this method accepts 3D-polynomial (X,Y,Z) or 3D-rational (X,Y,Z,Weight) trim curves. To do this, negate the absolute value of the first value in the VDim array. For example, if the first VDim value is 3, set it to -3. When you do this, 3D trim curves are expected.

If Dim is:

- 2, specify (X,Y) control points in each CtrlPointDbl sub-array.
- -2, specify (X,Y,Z) control points in each CtrlPointDbl sub-array.
- 3, specify (X,Y,Weight) control points in each CtrlPointDbl sub-array.
- -3, specify (X,Y,Z,Weight) control points in each CtrlPointDbl sub-array.

Multiplicity of a knot is the number of times it is repeated in the knot vector. There are specific rules around number of knots, knot multiplicity, and curve order. For example, if a curve's Periodic value is:

- 0 (non-periodic), there must be (NumCtrlPoints + Order) knots and the maximum multiplicity of any knot is Order.
- 1 (periodic), there must be (NumCtrlPoints + 1) knots and the maximum multiplicity of any knot is Order. If a knot has multiplicity greater than 1, repetitions must occur at the end of the knot vector.

If you do not want to specify a UV range, use null or Nothing in the UvRange array elements.

After calling this method to add a trimming loop, you can generate a trimmed surface by calling [IBody2::CreateTrimmedSurface](#). You can create a body from trimmed surfaces using [IBody2::CreateBodyFromSurfaces](#).

NOTE: It is highly recommended that you consult other resources for the mathematics behind b-splines, knots, knot multiplicity, and control points before attempting to use this method.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::AddTrimmingLoop2 Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

IConeParams Property (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IConeParams Property (ISurface)

Gets the parameters of a conical surface.

.NET Syntax

Visual Basic (Declaration)

```
ReadOnly Property IConeParams As System.Double
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As System.Double  
  
value = instance.IConeParams
```

C#

```
System.double IConeParams {get;}
```

C++/CLI

```
property System.double IConeParams {  
    System.double get();  
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

Array of doubles describing the parameters of a conical surface

Visual Basic for Applications (VBA) Syntax

See [Surface::IConeParams](#).

Remarks

Returns an array of 8 doubles:

- *origin.x*
- *origin.y*
- *origin.z*
- *axis.x*
- *axis.y*
- *axis.z*
- *radius*
- *half angle*

The *half angle*, *origin*, and *radius* parameters are in meters.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::ConeParams Property](#)
[ISurface::IsCone Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ICopy Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICopy Method (ISurface)

Gets a copy of this surface.

.NET Syntax

Visual Basic (Declaration)

```
Function ICopy() As Surface
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As Surface  
  
value = instance.ICopy()
```

C#

```
Surface ICopy()
```

C++/CLI

```
Surface^ ICopy();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Copy of [surface](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::ICopy](#).

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::Copy Method](#)
[ISurface::CreateNewCurve Method](#)
[ISurface::ICreateNewCurve Method](#)
[ISurface::IMakeIsoCurve Method](#)
[ISurface::IMakeIsoCurves Method](#)
[ISurface::MakeIsoCurve Method](#)
[ISurface::MakeIsoCurves Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ICreateNewCurve Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICreateNewCurve Method (ISurface)

Creates a new empty curve for the part.

.NET Syntax

Visual Basic (Declaration)

```
Function ICreateNewCurve() As Curve
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim value As Curve  
  
value = instance.ICreateNewCurve()
```

C#

```
Curve ICreateNewCurve()
```

C++/CLI

```
Curve^ ICreateNewCurve();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Newly created [curve](#)

Visual Basic for Applications (VBA) Syntax

See [Surface::ICreateNewCurve](#).

Remarks

The intention is that with a handle on such (initially empty) curves, appropriate construction can be called that eventually amounts to non-trivial objects.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::CreateNewCurve Method](#)
[ISurface::IMakeIsoCurve Method](#)
[ISurface::IMakeIsoCurves Method](#)
[ISurface::IMakeIsoCurve Method](#)
[ISurface::IMakeIsoCurves Method](#)
[ICurve::CreateTrimmedCurve2 Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

ICreateTrimmedSheet Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICreateTrimmedSheet Method (ISurface)

Obsolete. Superseded by [ISurface::ICreateTrimmedSheet4](#).

.NET Syntax

Visual Basic (Declaration)

```
Function ICreateTrimmedSheet( _  
    ByVal NCurves As System.Integer, _  
    ByRef Curves As Curve _  
) As Body
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim NCurves As System.Integer  
Dim Curves As Curve  
Dim value As Body  
  
value = instance.ICreateTrimmedSheet(NCurves, Curves)
```

C#

```
Body ICreateTrimmedSheet(  
    System.int NCurves,  
    ref Curve Curves  
)
```

C++/CLI

```
Body^ ICreateTrimmedSheet(  
& System.int NCurves,  
& Curve^% Curves  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NCurves

Curves

See Also

ICreateTrimmedSheet2 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICreateTrimmedSheet2 Method (ISurface)

Obsolete. Superseded by [ISurface::ICreateTrimmedSheet4](#).

.NET Syntax

Visual Basic (Declaration)

```
Function ICreateTrimmedSheet2( _  
    ByVal NCurves As System.Integer, _  
    ByRef Curves As Curve _  
    ) As Body2
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim NCurves As System.Integer  
Dim Curves As Curve  
Dim value As Body2  
  
value = instance.ICreateTrimmedSheet2(NCurves, Curves)
```

C#

```
Body2 ICreateTrimmedSheet2(  
    System.int NCurves,  
    ref Curve Curves  
)
```

C++/CLI

```
Body2^ ICreateTrimmedSheet2(  
& System.int NCurves,  
& Curve^% Curves  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NCurves

Curves

See Also

ICreateTrimmedSheet4 Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICreateTrimmedSheet4 Method (ISurface)

Obsolete. Superseded by [ISurface::CreateTrimmedSheet5](#).

.NET Syntax

Visual Basic (Declaration)

```
Function ICreateTrimmedSheet4( _  
    ByVal NCurves As System.Integer, _  
    ByRef Curves As Curve, _  
    ByVal PreserveAnalyticCurves As System.Boolean _  
    ) As Body
```

Visual Basic (Usage)



```

Dim instance As ISurface
Dim NCurves As System.Integer
Dim Curves As Curve
Dim PreserveAnalyticCurves As System.Boolean
Dim value As Body

value = instance.ICreateTrimmedSheet4(NCurves, Curves, PreserveAnalyticCurves)

```

C#

```

Body ICreateTrimmedSheet4(
    System.int NCurves,
    ref Curve Curves,
    System.bool PreserveAnalyticCurves
)

```

C++/CLI

```

Body^ ICreateTrimmedSheet4(
    & System.int NCurves,
    & Curve^% Curves,
    & System.bool PreserveAnalyticCurves
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

NCurves

Number of curves in the array of curves

Curves

Array of [curves](#) that represent the boundary of the sheet

PreserveAnalyticCurves

True to preserve analytic curves, false to store all trimming curves as SP-curves

Return Value

Newly created sheet [body](#)

Remarks

The array of curves represents all of the curves required to add the appropriate trimming loops to the surface. A NULL entry in the array represents the separation between loops.

The curves supplied are assumed to lie on the surface. If the curves are 2D curves, then they should be created using this surface.

If your application is creating a trimmed sheet body from an input periodical surface without trimming curves, then the curve array may be empty.

Before calling this method, trim the curves created by [IModeler::ICreateArc](#) and [IModeler::ICreateLine](#) by calling [ICurve::CreateTrimmedCurve2](#). Elliptical curves created by [IModeler::ICreateEllipse](#) do not need to be trimmed before calling this method.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::CreateTrimmedSheet4 Method](#)

Availability

SOLIDWORKS 2008 FCS, Revision Number 16.0

SOLIDWORKS API Help

[Send comments on this topic.](#)

ICylinderParams Property (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : ICylinderParams Property (ISurface)

Gets the parameters of a cylindrical surface.

.NET Syntax

Visual Basic (Declaration)

```
ReadOnly Property ICylinderParams As System.Double
```

Visual Basic (Usage)

```
Dim instance As ISurface
Dim value As System.Double

value = instance.ICylinderParams
```

C#

```
System.double ICylinderParams {get;}
```

C++/CLI

```
property System.double ICylinderParams {
    System.double get();
}
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Property Value

Array of doubles describing the parameters of a cylindrical surface

Visual Basic for Applications (VBA) Syntax

See [Surface::ICylinderParams](#).

Remarks

Returns an array of 7 doubles:

- *origin.x*
- *origin.y*
- *origin.z*
- *axis.x*
- *axis.y*
- *axis.z*
- *radius*

The *origin* and *radius* parameters are in meters.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::CylinderParams Property](#)
[ISurface::IsCylinder Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

Identity Method (ISurface)

[See Also](#) [Example](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : Identity Method (ISurface)

Gets the type of surface.

.NET Syntax

Visual Basic (Declaration)

```
Function Identity() As System.Integer
```

Visual Basic (Usage)

```
Dim instance As ISurface
Dim value As System.Integer

value = instance.Identity()
```

C#

```
System.int Identity()
```

C++/CLI

```
System.int Identity();
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Return Value

Type of surface as defined in [swSurfaceTypes_e](#)

Visual Basic for Applications (VBA) Syntax

See [Surface.Identity](#).

Example

[Get Surface Type \(VBA\)](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

Evaluate Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IEvaluate Method (ISurface)

Evaluates the surface, given the u and v parameters of the surface.

.NET Syntax

Visual Basic (Declaration)

```
Function IEvaluate( _  
    ByVal UParam As System.Double, _  
    ByVal VParam As System.Double, _  
    ByVal NumUDeriv As System.Integer, _  
    ByVal NumVDeriv As System.Integer _  
    ) As System.Double
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim UParam As System.Double  
Dim VParam As System.Double  
Dim NumUDeriv As System.Integer  
Dim NumVDeriv As System.Integer  
Dim value As System.Double  
  
value = instance.IEvaluate(UParam, VParam, NumUDeriv, NumVDeriv)
```

C#

```
System.double IEvaluate(  
    System.double UParam,  
    System.double VParam,  
    System.int NumUDeriv,  
    System.int NumVDeriv  
)
```

C++/CLI

```
System.double IEvaluate(  
& System.double UParam,  
& System.double VParam,  
& System.int NumUDeriv,  
& System.int NumVDeriv  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

UParam
Value of u parameter

VParam
Value of v parameter

NumUDeriv
Number of u derivatives required

NumVDeriv
Number of v derivatives required

Visual Basic for Applications (VBA) Syntax

See [Surface::IEvaluate](#).

Remarks

This method returns the evaluated point, normal, and derivatives with respect to u and v up to order NumUDerivs and NumVDerivs, respectively. If NumUDerivs and NumVDerivs are 0, then no derivatives are returned.

For more information on the valid u and v parameters of the surface, see [ISurface::Parameterization](#) or [ISurface::IParameterization](#). Also, use [IFace2::FaceInSurfaceSense](#) to check to see if the normal of the face is in the same direction or in the opposite direction of the surface normal.

The evaluation returns the following array of (3 * (((NumUDerivs +1) * (NumVDerivs + 1)) + 1)) doubles:

[evaluatedPoint[3], evaluatedDerivatives[xx], evaluatedNormal[3]]

where:

evaluatedPoint[3] = point representing the evaluated X,Y,Z point in meters

evaluatedDerivatives[xx] = array of vectors representing the derivatives

where the *n*th derivative wrt u (i <= NumUDerivs)

and the *n*th derivative wrt v (j <= NumVDerivs)

would be vector number (i + (NumUDerivs + 1) * j) - 1 in the *evaluatedDerivatives* array.

If NumUDerivs and NumVDerivs are 0, then no derivatives are returned.

evaluatedNormal[3] = a vector representing the evaluated normal

The following table describes the number of derivatives that can be requested based on the surface type. If you request more derivatives than allowed, then this method returns all data as zeros. To determine the surface type, see [ISurface::Identity](#).

		U Derivatives		
		0	1	2
	0	a	a	b
V Derivatives	1	a	a	c
	2	b	c	d

where:

- a - All surface types
- b - All surface types except blend surfaces
- c - All surface types except blend surfaces and offset surfaces
- d - All surface types except blend surfaces but offset surfaces will only return 0th, 1st, and 2nd derivatives (the uncalculated 3rd and 4th derivatives are returned as 0 vectors).

For example, if you specified 2 derivatives in both u and v, then an array containing 10 vectors is returned. The returned data is in model space coordinates.

[P(u,v) P(u,v)/du P(u,v)/dudu]
[P(u,v)/dv P(u,v)/dudv P(u,v)/dududv]
[P(u,v)/dvdv P(u,v)/dudvdv P(u,v)/dududvdv]
[N(u,v)]

where:

- du the partial derivative in the u direction
- dv the partial derivative in the v direction
- N(u,v) the normal vector
- Each P() represents a model space point or vector depending on the entry in the array

If you request 2 derivatives for u and 1 for v, then an array containing 7 vectors is returned:

$[P(u,v)]$	$P(u,v)/du$	$P(u,v)/dudu$
$[P(u,v)/dv]$	$P(u,v)/dudv$	$P(u,v)/dududv$
$[N(u,v)]$		

If u = 1 and v = 2, then:

$[P(u,v)]$	$P(u,v)/du$
$[P(u,v)/dv]$	$P(u,v)/dudv$
$[P(u,v)/dv dv]$	$P(u,v)/dudv dv]$
$[N(u,v)]$	

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::Evaluate Method](#)
[ISurface::EvaluateAtPoint Method](#)
[ISurface::EvaluateAtPoint Method](#)
[ISurface::IReverseEvaluate Method](#)
[ISurface::ReverseEvaluate Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

IEvaluateAtPoint Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IEvaluateAtPoint Method (ISurface)

Evaluates a surface at the specified XYZ point.

.NET Syntax

Visual Basic (Declaration)

```
Function IEvaluateAtPoint( _  
    ByVal PositionX As System.Double, _  
    ByVal PositionY As System.Double, _  
    ByVal PositionZ As System.Double _  
    ) As System.Double
```

Visual Basic (Usage)



```
Dim instance As ISurface  
Dim PositionX As System.Double  
Dim PositionY As System.Double  
Dim PositionZ As System.Double  
Dim value As System.Double  
  
value = instance.IEvaluateAtPoint(PositionX, PositionY, PositionZ)
```

C#

```
System.double IEvaluateAtPoint(  
    System.double PositionX,  
    System.double PositionY,  
    System.double PositionZ  
)
```

C++/CLI

```
System.double IEvaluateAtPoint(  
& System.double PositionX,  
& System.double PositionY,  
& System.double PositionZ  
)
```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code.](#)

Parameters

PositionX
X position
PositionY
Y position
PositionZ

Z position

Return Value

Array of doubles (see [Remarks](#))

Visual Basic for Applications (VBA) Syntax

See [Surface::IEvaluateAtPoint](#).

Remarks

This method calculates the normal, the principal directions, and the principal curvatures, of the surface at the specified point.

Use [IFace2::FaceInSurfaceSense](#) to check the directions of the face normal and surface normal. [IFace2::FaceInSurfaceSense](#) returns true when the face normal and surface normal point in opposite directions, and false when they point in the same direction.

The return value is the following array of eleven doubles:

[*surfNorm*[*i*, *j*, *k*], *principalDir1*[*i*, *j*, *k*], *principalDir2*[*i*, *j*, *k*], *principalCurvature1*, *principalCurvature2*]

where:

surfNorm[*i*, *j*, *k*] = normalized vector describing the surface normal

principalDir1[*i*, *j*, *k*] = normalized vector describing the first principal direction

principalDir2[*i*, *j*, *k*] = normalized vector describing the second principal direction

principalCurvature1 = first principal curvature

principalCurvature2 = second principal curvature

Principal Curvature 1 is the minimum normal curvature at the point (largest radius). Principal Curvature 2 is the maximum normal curvature at the point.

The tangent direction producing Principal Curvature 1 is called the first principal direction, and the tangent direction producing Principal Curvature 2 is called the second principal direction.

It is a property of differentiable surfaces that *principalDir1* and *principalDir2* are orthogonal.

A positive curvature by convention implies a centre of curvature on the side pointed away from by the surface normal (convex).

See "Faux and Pratt Computational Geometry for Design and Manufacture" for more information.

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface::EvaluateAtPoint Method](#)
[ISurface::Evaluate Method](#)
[ISurface::IEvaluate Method](#)
[ISurface::IParameterization Method](#)
[ISurface::IReverseEvaluate Method](#)
[ISurface::Parameterization Method](#)
[ISurface::ReverseEvaluate Method](#)

SOLIDWORKS API Help

[Send comments on this topic.](#)

IFindMinimumRadius Method (ISurface)

[See Also](#)



[SolidWorks.Interop.sldworks Namespace](#) > [ISurface Interface](#) : IFindMinimumRadius Method (ISurface)

Gets the minimum radius of curvature for the selected surface.

.NET Syntax

Visual Basic (Declaration)

```
Function IFindMinimumRadius( _  
    ByRef UBound As System.Double, _  
    ByRef VBound As System.Double, _  
    ByRef NumOfRadius As System.Integer, _  
    ByRef Radius As System.Object, _  
    ByRef Location As System.Object, _  
    ByRef UVParameter As System.Object _  
) As System.Boolean
```

Visual Basic (Usage)

```
Dim instance As ISurface  
Dim UBound As System.Double  
Dim VBound As System.Double
```



```

Dim NumOfRadius As System.Integer
Dim Radius As System.Object
Dim Location As System.Object
Dim UVParameter As System.Object
Dim value As System.Boolean

```

```

value = instance.IFindMinimumRadius(UBound, VBound, NumOfRadius, Radius, Location, UVParameter)

```

C#

```

System.bool IFindMinimumRadius(
    ref System.double UBound,
    ref System.double VBound,
    out System.int NumOfRadius,
    out System.object Radius,
    out System.object Location,
    out System.object UVParameter
)

```

C++/CLI

```

System.bool IFindMinimumRadius(
    & System.double% UBound,
    & System.double% VBound,
    & [Out] System.int NumOfRadius,
    & [Out] System.Object^ Radius,
    & [Out] System.Object^ Location,
    & [Out] System.Object^ UVParameter
)

```

NOTE: See [Differences Between Unmanaged C++ and C++/CLI Code](#).

Parameters

[UBound](#)
 MinMax UParameter
[VBound](#)
 MinMax VParameter
[NumOfRadius](#)
 Number of radius; can be 0, 1, or 2
[Radius](#)
 Minimum radius of curvature (see **Remarks**)
[Location](#)
[Position](#) where the minimum radius curvature occurred (see **Remarks**)
[UVParameter](#)
[UV parameters](#); because points are UV, third ordinates are 0 (see **Remarks**)

Return Value

True if operation succeeds, false if it fails

Visual Basic for Applications (VBA) Syntax

See [Surface.IFindMinimumRadius](#).

Example

In-process, unmanaged C++:

```

////////////////////////////////////
// IMinRadius implementation
STDMETHODIMP CMinRadius::StartNotepad()
{
    // TODO: Add your implementation code here
    CComPtr<IModelDoc> iModelDoc2;
    m_isldWorks->get_IActiveDoc(&iModelDoc2);

    CComPtr<ISelectionMgr> swSelMgr;
    iModelDoc2->get_ISelectionManager(&swSelMgr);
    struct IDispatch* pDispSelection;

```

```

CComPtr<IFace2> swFace;
swSelMgr->GetSelectedObject6(1, -1, &pDispSelection);
pDispSelection->QueryInterface(__uuidof(IFace2), reinterpret_cast<void**>(&swFace));
pDispSelection->Release();

double boundArr[4];
swFace->IGetUVBounds ( boundArr );
double UBound[2] = {boundArr[0], boundArr[1]};
double VBound[2] = {boundArr[2], boundArr[3]};

CComPtr<ISurface> swSurface;
swFace->IGetSurface(&swSurface);

long numOfRadius(0);
VARIANT radius, Location, UVParameter;
VARIANT_BOOL status(FALSE);
swSurface->IFindMinimumRadius(UBound, VBound, &numOfRadius, &radius, &Location, &UVParameter, &status);
SafeDoubleArray radiusSA(radius);

double test1 = radiusSA[0];
double test2 = radiusSA[1];

SafeDISPATCHArray locationSA(Location);
CComPtr<IMathPoint> swMathPoint1;
CComPtr<IMathPoint> swMathPoint2;
locationSA[0]->QueryInterface(__uuidof(IMathPoint), reinterpret_cast<void**>(&swMathPoint1));
locationSA[1]->QueryInterface(__uuidof(IMathPoint), reinterpret_cast<void**>(&swMathPoint2));

double locationArra1[3];
double locationArra2[3];
swMathPoint1->get_IArrayData(locationArra1);
swMathPoint2->get_IArrayData(locationArra2);

SafeDISPATCHArray UVParameterSA(UVParameter);
return S_OK;
}

```

Remarks

The search is confined to the portion of the selected curve lying inside of UBound and VBound.

COM returns these data types for these parameters:

- Radius: VARIANT of SafeDoubleArray
- Location: VARIANT of SafeDispatchArray of [IMathpoint](#)
- UVParameter: VARIANT of SafeDispatchArray of [IMathpoint](#)

See Also

[ISurface Interface](#)
[ISurface Members](#)
[ISurface.FindMinimumRadius Method](#)
[IFace2.IGetUVBounds Method](#)
[IFace2.IGetUVBounds Method](#)
[ICurve.FindMinimumRadius Method](#)
[ICurve.FindMinimumRadius Method](#)
[ISurface.Parameterization Method](#)
[ISurface.Parameterization Method](#)

Availability

SOLIDWORKS 2009 FCS, Revision Number 17.0